

Anatomy of a Perfect GNS3 Project

A First-Principles Specification and Definition of Done Checklist

Based on GNS3 Server v2.2.x Source Code Analysis
File Format Revision 9 | .gns3project Portable Project Format

This document provides a comprehensive, source-code-backed analysis of the GNS3 .gns3project file format. It deconstructs the JSON schema, node-specific constraints, link mapping rules, ZIP archive structure, and common failure modes. Each finding is traced directly to the GNS3 Server source code (controller/project.py, controller/topology.py, schemas/topology.py, controller/node.py, controller/ports/port_factory.py, and the individual node-type schemas). The document concludes with a strict "Definition of Done" checklist that any GNS3 project generator must satisfy to produce files that are 100% compatible with any GNS3 Server instance.

Table of Contents

1. Schema Determinism and Mandatory Fields	3
1.1 Top-Level Mandatory Keys	3
1.2 UUID Format Requirements	3
1.3 Topology Object Hierarchy	4
2. Node-Specific Constraints (Hardware Logic)	4
2.1 Dynamips (Cisco IOS Routers)	4
2.2 VPCS (Virtual PC Simulator)	5
2.3 Ethernet Switch	5
2.4 QEMU and Docker	5
3. Link and Interface Mapping Rules	6
3.1 Link Structure and Required Fields	6
3.2 How GNS3 Reconciles Links with Ports	6
3.3 Consequences of Invalid Port References	6
4. ZIP Structure and Internal File Dependencies	7
4.1 Archive Layout	7
4.2 Config File Pathing Conventions	7
4.3 Relative vs. Absolute Paths	8
5. Failure Modes and Edge Cases	8
5.1 Image MD5 Mismatch or Missing Image	8
5.2 Port/Adapter Number Mismatch in Links	8
5.3 UUID Collisions Across Projects	9
5.4 Revision Version Mismatch	9
5.5 Invalid Slot Module for Platform	9
6. Definition of Done Checklist	9

1. Schema Determinism and Mandatory Fields

The GNS3 Server validates every `.gns3project` file against a strict JSON Schema defined in `gns3server/schemas/topology.py`. This schema is enforced during both import (`import_project.py`) and project loading (`topology.py::_check_topology_schema()`). Understanding the exact mandatory fields is the foundation for generating a valid project file. The schema uses JSON Schema Draft-04 with `additionalProperties: False` on most objects, meaning any key not explicitly declared in the schema will cause validation to fail. This makes the format strictly deterministic: there is no room for undeclared extensions at the topology level.

1.1 Top-Level Mandatory Keys

The root JSON object of a `.gns3` file (or the `project.gns3` file inside a `.gns3project` ZIP) must contain the following six keys. The schema declares them as "required", and their absence triggers an immediate `HTTPConflict` error with the message "the project.gns3 file is corrupted". This validation occurs in `import_project.py` lines 24-33 where the server first attempts to parse the JSON and extract `topology["project_id"]` and `topology["name"]` before any further processing.

Key	Type	Description	Source
project_id	string (UUID)	Project UUID, 36 chars, regex-validated	schemas/topology.py
type	"topology"	Literal string, always "topology"	schemas/topology.py
revision	integer	File format version (current: 9)	topology.py GNS3_FILE_FORMAT_REVISION
version	string	GNS3 software version that wrote the file	schemas/topology.py
name	string	Human-readable project name	schemas/topology.py
topology	object	Container for nodes, links, drawings, computes	schemas/topology.py

Table 1. Top-level mandatory keys in a `.gns3` file

1.2 UUID Format Requirements

All three identifier types in the GNS3 topology share the same format constraint, defined in `schemas/topology.py`, `schemas/node.py`, and `schemas/link.py`. They must match the regex pattern `^[a-fA-F0-9]{8}-[a-fA-F0-9]{4}-[a-fA-F0-9]{4}-[a-fA-F0-9]{4}-[a-fA-F0-9]{12}$`. This is the standard UUID v4 string representation (lowercase or uppercase hex digits accepted). The controller's Project class constructor in `controller/project.py` explicitly validates this by calling `UUID(project_id, version=4)` and raising `HTTPBadRequest` if it fails. While the schema regex does not enforce the RFC 4122 version bits (the "4" in the third group), the controller's runtime validation does. In practice, always generate proper UUIDv4 values using `uuid.uuid4()` or equivalent.

Identifier	Schema Key	Pattern	Validation
------------	------------	---------	------------

project_id	topology.project_id	36-char UUID regex	UUID(v, version=4) in controller
node_id	node.node_id	36-char UUID regex	uuid.uuid4() default in Node.__init__
link_id	link.link_id	36-char UUID regex	uuid.uuid4() at link creation
drawing_id	drawing.drawing_id	36-char UUID regex	uuid.uuid4() at drawing creation

Table 2. UUID identifier types and their validation

1.3 Topology Object Hierarchy

The "topology" object is the core container. Its schema in `schemas/topology.py` declares exactly four required arrays: "nodes", "links", "drawings", and "computes". The `additionalProperties: False` constraint means no other keys are permitted inside the topology object. Each array has its own item schema: nodes use `NODE_OBJECT_SCHEMA`, links use `LINK_OBJECT_SCHEMA`, drawings use `DRAWING_OBJECT_SCHEMA`, and computes use `COMPUTE_OBJECT_SCHEMA`. A valid .gns3 file may have empty arrays (e.g., no drawings, no computes), but the arrays themselves must be present. Omitting any of the four arrays causes schema validation to fail with a "required" error. The `project_to_topology()` function in `topology.py` always writes all four arrays when exporting a project.

2. Node-Specific Constraints (Hardware Logic)

Each `node_type` in GNS3 has a unique set of mandatory properties that the compute server expects when creating a node. The topology schema performs a two-pass validation: first the top-level node object is validated against `NODE_OBJECT_SCHEMA`, then for certain node types (currently only dynamips), the node's "properties" dict is validated against the compute-specific schema (`dynamips_vm.VM_CREATE_SCHEMA`). This second validation is performed in `topology.py::_check_topology_schema()` lines 37-48. The node-level schema requires only three keys: "name", "node_type", and "compute_id". Everything else is either optional or computed by the server.

2.1 Dynamips (Cisco IOS Routers)

Dynamips nodes have the most complex property requirements of any node type. The compute creation schema in `schemas/dynamips_vm.py` (exposed via the API docs at `/v2/compute/projects/{project_id}/dynamips/nodes`) declares two mandatory fields: "image" (path to the IOS image file) and "platform" (the Cisco router platform identifier). Additionally, "name" is mandatory at the node level. The platform determines which slot modules are valid, the maximum RAM, and the port naming convention. The `DynamipsPortFactory` in `controller/ports/port_factory.py` contains the `ADAPTER_MATRIX` dictionary that maps module names (e.g., "PA-FE-TX", "NM-16ESW") to their port types and counts. The WIC slots use a separate `WIC_MATRIX`. Slot keys in the properties dict follow the pattern `slot0`, `slot1`, etc., up to `slot6`. An empty or null slot value means no module is installed in that slot.

Property	Mandatory	Type	Description
----------	-----------	------	-------------

platform	Yes	string	e.g., "c7200", "c3745", "c2600"
image	Yes	string	Path to IOS image (filename only for portable projects)
ram	No*	integer	RAM in MB. Defaults per platform in PLATFORMS_DEFAULT_RAM
nvrn	No	integer	NVRAM in KB (default: 128 or 256 depending on platform)
slot0 - slot6	No	string/null	Network module name per slot (null = empty)
wic0 - wic2	No	string/null	WIC module name (WIC-1T, WIC-2T, WIC-1ENET)
chassis	No	string	For c3600 series: "3620", "3640", "3660"
console_type	No	enum	"telnet" or "none" (default: "telnet")
startup_config_content	No	string	IOS startup config text (inline, not file path)
idlepc	No	string	Idle-PC value for CPU optimization

Table 3. Dynamips node properties (*ram is required by API but defaults exist)

2.2 VPCS (Virtual PC Simulator)

VPCS is the simplest node type. The creation schema in `schemas/vpcs.py` requires only "name". The only other notable property is "startup_script", which accepts a string containing VPCS commands (e.g., "ip 10.0.0.1/24"). There are no hidden mandatory requirements. The console_type defaults to "telnet". The VPCS binary is expected to be available on the compute server at the path configured in the GNS3 server settings (typically `/usr/bin/vpcs`). VPCS nodes are always assigned a single Ethernet port (adapter_number=0, port_number=0) by the StandardPortFactory. They cannot have more than one network connection without inserting a switch between multiple VPCS instances and their upstream devices.

2.3 Ethernet Switch

Ethernet switches are unique because they use a "ports_mapping" array inside the properties dict to define their port configuration. The schema in `schemas/ethernet_switch.py` defines the port structure with three required keys per port: "name" (string, minLength 1), "port_number" (integer, minimum 0), and "type" (enum: "access", "dot1q", "qinq"). Optional keys include "vlan" (integer, minimum 1) and "ethertype" (for QinQ: "", "0x8100", "0x88A8", "0x9100", "0x9200"). Access ports require a vlan number; dot1q trunk ports use vlan as the native VLAN; qinq ports use ethertype to specify the outer tag protocol. The StandardPortFactory generates the port list from ports_mapping at runtime, and the controller node's `_list_ports()` method calls this factory lazily. The switch creation schema requires only "name". If "ports_mapping" is omitted, the switch starts with no ports configured, and ports are added dynamically as links are created.

2.4 QEMU and Docker

QEMU nodes have a rich schema in `schemas/qemu.py` with many optional properties for disk images, RAM, CPUs, and adapter configuration. The only mandatory fields are "name" and "qemu_path" or "hda_disk_image" (in practice, a disk image or kernel image is needed for the VM to boot). The "adapters" property (integer, 0-275) controls the number of network adapters. The "adapter_type" string specifies the virtual NIC model (e.g., "e1000", "virtio-net-pci"). Each disk image slot (hda, hdb, hdc, hdd) has a companion *_md5sum field for integrity verification. The "linked_clone" boolean (default: true for portable projects) determines whether the disk image is copied or linked.

Docker nodes, defined in `schemas/docker.py`, require "name" and "image" (the Docker image name, e.g., "alpine:latest"). The "adapters" property (integer, 0-99) sets the number of virtual NICs. The "start_command" overrides the container's CMD entry, and "environment" passes environment variables. Console types supported are "telnet", "vnc", "http", "https", and "none". A critical detail: Docker and QEMU nodes are automatically migrated to the GNS3 VM compute when importing a portable project on non-Linux hosts, as seen in `import_project.py` lines 83-89. This means their `compute_id` will be rewritten from "local" to "vm" during import.

3. Link and Interface Mapping Rules

3.1 Link Structure and Required Fields

A link in the GNS3 topology connects exactly two nodes. The `LINK_OBJECT_SCHEMA` in `schemas/link.py` defines the structure. The critical part is the "nodes" array, which must contain exactly two objects, each with three required fields: "node_id" (UUID of the connected node), "adapter_number" (integer, the slot/adapter index), and "port_number" (integer, the port index within that adapter). The "link_id" is a UUID that uniquely identifies the link. Optional fields include "link_type" (enum: "ethernet", "serial"), "suspend" (boolean), and "filters" (for packet filtering). Each node entry may also have a "label" object that defines the text displayed on the link endpoint in the GUI.

3.2 How GNS3 Reconciles Links with Ports

When the GNS3 controller loads a topology, it does not store port definitions in the JSON. Instead, ports are computed dynamically by the `Node._list_ports()` method, which calls the appropriate port factory. For Dynamips nodes, `DynamipsPortFactory` iterates over the sorted slot keys in the properties dict, looks up each module in the `ADAPTER_MATRIX`, and generates port objects with `adapter_number` equal to the slot index and `port_number` incrementing from 0 within each adapter. For standard devices (IOU, QEMU, Docker, VPCS, Ethernet Switch), `StandardPortFactory` generates ports based on the `ethernet_adapters` or `adapters` property, with `port_by_adapter` controlling how many ports per adapter (4 for IOU, 1 for others). When a link references an (adapter_number, port_number) pair, the controller looks up the corresponding port in the node's computed ports list. If the pair does not correspond to any defined port, the link will fail to activate when the node starts, and the GNS3 GUI will show a warning icon on the link.

3.3 Consequences of Invalid Port References

If a link references an `adapter_number` or `port_number` that does not exist in the node's computed port list, the behavior depends on the node type. For Dynamips, the error surfaces immediately when the router is started, because Dynamips tries to connect a NIO (Network Input/Output) to a non-existent slot/port, resulting in a "Dynamips error" with a message like "slot X does not exist" or "port Y does not exist in slot X". For IOU, the link may appear to connect in the GUI but packets will not flow through the undefined interface. For Ethernet switches, referencing a `port_number` beyond the `ports_mapping` array is silently ignored until a packet needs to traverse that port. The safest approach is to always verify that every (`adapter_number`, `port_number`) pair in every link matches a port that the port factory would generate for the given node properties.

4. ZIP Structure and Internal File Dependencies

4.1 Archive Layout

A `.gns3project` file is a standard ZIP archive. The import logic in `import_project.py` first attempts to read "project.gns3" from the ZIP root. If this key is missing, the import fails with "Cannot import project, project.gns3 file could not be found". After reading the JSON, the entire ZIP is extracted to the project directory using `zip_file.extractall(path)`. The export logic in `export_project.py` walks the project directory and writes all files using relative paths from the project root, preserving the directory structure inside the ZIP.

ZIP Path	Description	Required
project.gns3	The topology JSON file (renamed to .gns3 after import)	Yes (for import)
project-files/dynamips//	Dynamips working directory per node (NVRAM, configs)	No
project-files/vpcs//	VPCS working directory (startup scripts)	No
project-files/iou//	IOU working directory (nvram, startup configs)	No
project-files/qemu//	QEMU working directory (disk images, snapshots)	No
project-files/docker//	Docker working directory (persistent files)	No
project-files/ethernet_switch//	Ethernet switch working directory	No
images/	Bundled OS images (when <code>include_images=True</code> during export)	No
snapshots/	Project snapshots (when <code>include_snapshots=True</code>)	No

Table 4. ZIP archive directory structure

4.2 Config File Pathing Conventions

Startup and private configuration files are stored in the node's working directory under `project-files/`. The naming convention is `startup-config.cfg` for IOS startup configs and `private-config.cfg` for IOS private configs. For VPCS, the startup script is stored as `startup.vpc`. These files are referenced in the JSON in two different ways depending on the node type. For Dynamips nodes, the properties dict can contain `"startup_config_content"` (the actual config text as a string) which GNS3 writes to the `startup-config.cfg` file upon node creation. Alternatively, the config file can exist in the ZIP at the expected path and GNS3 will read it from there. The key insight is that `startup_config_content` is an inline property that gets written to disk, while the actual `.cfg` file in the ZIP is the runtime artifact. For portable projects, the inline content approach is preferred because it is self-contained within the JSON and does not depend on external file paths.

4.3 Relative vs. Absolute Paths

The GNS3 export process in `export_project.py` rewrites all `compute_id` values to `"local"` (unless `keep_compute_ids=True`) and converts absolute image paths to relative paths. The `_patch_project_file()` function processes the `.gns3` JSON before adding it to the ZIP, ensuring that image paths reference just the filename (e.g., `"c7200-adventerprisek9-mz.124-24.T5.image"`) rather than the full filesystem path (e.g., `"/opt/gns3/images/IOS/c7200-adventerprisek9-mz.124-24.T5.image"`). When images are bundled with the project (`include_images=True`), they are placed in the `images/` directory inside the ZIP and the import process copies them to the server's images directory. Any absolute path in the properties dict will be resolved relative to the GNS3 images directory on the importing server. This means that for maximum portability, all image references should use filenames only, not absolute paths.

5. Failure Modes and Edge Cases

Even a structurally valid JSON file can fail to import or boot nodes in the GNS3 GUI. The following are the top five reasons, each traced to specific source code paths and error handling logic in the GNS3 server.

5.1 Image MD5 Mismatch or Missing Image

When a Dynamips node is created, the compute server checks the IOS image file against the `image_md5sum` field if provided. If the checksum does not match, the node creation fails. More commonly, if the image file itself does not exist on the compute server at the expected path, the server raises an `ImageMissingError`. The controller's `Node.create()` method in `controller/node.py` catches this specific exception and attempts to upload the missing image from the project files (up to 6 retries). If the image is neither on the compute server nor in the project ZIP, the node will be created in a broken state and cannot be started. For QEMU nodes, each disk image has a corresponding `*_md5sum` field, and the same missing-image retry logic applies.

5.2 Port/Adapter Number Mismatch in Links

As detailed in Section 3, a link that references a non-existent (`adapter_number`, `port_number`) pair will cause a runtime error when the node starts. For Dynamips, this manifests as a "Dynamips error" when the

NIO creation command fails. The error is not caught during import; it only surfaces when the user attempts to start the node. This is particularly dangerous because the topology appears valid in the GUI (the link is drawn between the two nodes) but silently fails at runtime. The only way to detect this proactively is to cross-reference every link's port references against the ports that the port factory would generate for the given node properties.

5.3 UUID Collisions Across Projects

When importing a .gns3project, the server regenerates all UUIDs (project_id, node_id, link_id, drawing_id) to avoid collisions with existing projects. This is done by `regenerate_topology_ids()` in `import_project.py`. However, if a project is loaded directly (not imported as a portable project), the UUIDs are used as-is. If two projects share the same project_id, the second load will fail or overwrite the first. Similarly, if a node_id within a project is duplicated, the controller will silently use only one of the nodes. The controller uses a dict keyed by node_id to store nodes, so duplicate node_ids cause data loss without any error message. Always ensure UUIDs are unique within a project and, for portable projects, let the import process regenerate them.

5.4 Revision Version Mismatch

The `load_topology()` function in `topology.py` checks the revision field against `GNS3_FILE_FORMAT_REVISION` (currently 9). If the file's revision is greater than 9, the server refuses to load it with the message "This project was created with more recent version of GNS3". If the revision is missing or less than 9, the server attempts to apply migration patches from older formats (revisions 1-8). These migration paths are complex and may not handle all edge cases. Setting revision=9 and version to the current GNS3 version string is the safest approach. Never set revision to a future value.

5.5 Invalid Slot Module for Platform

The `DynamipsPortFactory.ADAPTER_MATRIX` in `controller/ports/port_factory.py` contains a fixed list of valid module names. If a node's properties dict specifies a slot value that is not in this matrix (e.g., "NM-1T" which does not exist in the `ADAPTER_MATRIX`), the port factory will raise a `KeyError` when computing ports, which propagates as an `HTTPConflict` error. This is not caught during schema validation because the schema only validates that slot values are strings; it does not check them against the `ADAPTER_MATRIX`. The same applies to WIC modules: only "WIC-1ENET", "WIC-1T", and "WIC-2T" are valid. Additionally, not all modules are compatible with all platforms. For example, PA-A1 (ATM port adapter) only works in the c7200 series, while NM-16ESW only works in the c2600/c3600 series. Specifying an incompatible module will cause Dynamips to fail at node start time.

6. Definition of Done Checklist

The following checklist defines the exact criteria that a generated .gns3project file must satisfy to be 100% compatible with any GNS3 Server v2.2.x instance. Each item is derived from the source-code analysis in the preceding sections. A project generator must pass ALL items to be considered compliant.

ID	Requirement	Source
S1	Root JSON contains all 6 mandatory keys: project_id, type, revision, version, name, topology	schemas/topology.py "required"
S2	project_id is a valid UUIDv4 string matching the 36-char regex pattern	controller/project.py UUID validation
S3	revision = 9 (GNS3_FILE_FORMAT_REVISION constant)	topology.py line 31
S4	type = "topology" (literal string, no other value accepted)	schemas/topology.py enum
S5	version = current GNS3 version string (e.g., "2.2.52")	topology.py __version__
S6	topology object contains all 4 required arrays: nodes, links, drawings, computes	schemas/topology.py "required"
S7	topology has no extra keys (additionalProperties: False)	schemas/topology.py
N1	Every node has: name (string), node_type (valid enum), compute_id (string)	schemas/node.py "required"
N2	node_id is a valid UUIDv4 for every node	schemas/node.py pattern
N3	node_type is one of the 14 valid enum values	schemas/node.py NODE_TYPE_SCHEMA

ID	Requirement	Source
N4	Dynamips nodes: properties contain "platform" and "image"	dynamips_vm.VM_CREATE_SCHEMA "required"
N5	Dynamips slot values are from ADAPTER_MATRIX only (no phantom modules)	port_factory.py ADAPTER_MATRIX
N6	Dynamips WIC values are from WIC_MATRIX only	port_factory.py WIC_MATRIX
N7	Dynamips c3600 nodes include "chassis" property ("3620","3640","3660")	Dynamips chassis handling
N8	Docker nodes: properties contain "image" (Docker image name)	docker.py DOCKER_CREATE_SCHEMA
N9	No read-only properties in node dict: ports, status, id, project, compute, host, links	controller/node.py CONTROLLER_ONLY_PROPERTIES
L1	Every link has a link_id (valid UUIDv4)	schemas/link.py pattern
L2	Every link.nodes array has exactly 2 entries	schemas/link.py "nodes" array
L3	Each link node entry has: node_id, adapter_number, port_number	schemas/link.py "required"
L4	All (adapter_number, port_number) pairs in links match computed ports	port_factory.py cross-reference

ID	Requirement	Source
----	-------------	--------

L5	No parallel links between the same node pair	GNS3 link validation
Z1	ZIP contains "project.gns3" at root level	import_project.py line 24
Z2	Config files under project-files///	export_project.py directory walk
Z3	Image paths are filenames only (no absolute paths) for portability	export_project.py _patch_project_file
Z4	compute_id set to "local" for portable projects	export_project.py compute rewriting
Z5	No startup_config_content or private_config_content in node properties dict (use ZIP files instead, or strip before export)	export_project.py property stripping
F1	All referenced IOS/QEMU images exist on target server or are bundled in ZIP	ImageMissingError handling
F2	No UUID collisions within a single project	controller dict key uniqueness
F3	Slot modules are platform-compatible (PA-* for c7200, NM-* for c2600/c3600/c3700)	Dynamips platform constraints
F4	No node_type values that the profile blocks (e.g., IOU when supports_iou=False)	preflight filtering

ID	Requirement	Source
F5	Ethernet switch ports_mapping entries have required: name, port_number, type	ethernet_switch.py schema

Table 5. Definition of Done Checklist (S=Schema, N=Node, L=Link, Z=ZIP, F=Failure Prevention)